Practical - 5

Practical Title: Registration Form with Frontend Validation and User-Friendly Error Handling

Problem Definition: Create a student registration form with HTML5 input types and JavaScript validation for name, email, mobile number, password, confirm password, course, year, gender, and terms acceptance. Use Regular Expression for validations.

A. Theory & Analysis

1. Are correct input types and attributes used?

Yes. The form uses HTML5 input types: type='text' for name, type='email' for university email, type='tel' for mobile number, type='password' for passwords, select dropdowns for course and academic year, radio buttons for gender, and type='checkbox' for terms acceptance. Each field has required attributes and label associations.

2. Are validation errors displayed near the relevant fields?

Yes. Validation errors appear inline directly under each input field using small red error text elements (such as #emailError, #mobileError). If an entry fails validation, the error message clarifies what is needed, and form submission is prevented.

3. Is password strength checked?

Yes. While the form submission regex requires at least 6 characters with a letter and a digit (/^(?=.*[A-Za-z])(?=.*[0-9]).{6,}$/), a separate real-time input event evaluates password strength dynamically: under 6 characters shows 'Weak', adding uppercase letters and digits updates the meter to 'Strong', and others show 'Medium'.

4. Is the form accessible using keyboard and screen-reader-friendly labels?

Yes. Form controls have explicit <label for='inputId'> tags, logical tab order for keyboard navigation, and visible error messages that explain the issue clearly.

B. Implementation Details

I built the student registration form in pages/register.html with client-side validation in js/register.js. The script validates input fields with regular expressions: Name (/^[A-Za-z]{2,50}$/), Email (/^[^\s@]+@[^\s@]+\.[^\s@]+$/), Mobile (/^[6-9][0-9]{9}$/), and Password (/^(?=.*[A-Za-z])(?=.*[0-9]).{6,}$/). The script tests matching confirm passwords, ensures terms are accepted, and shows real-time password strength feedback.

C. Evidence of Work

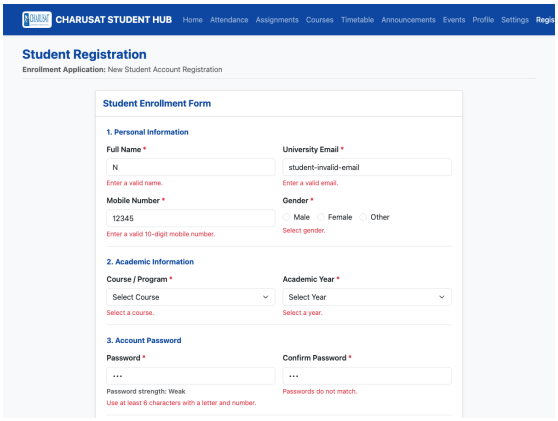


Figure 1: Validation Error States Showing Inline Feedback for Malformed Inputs

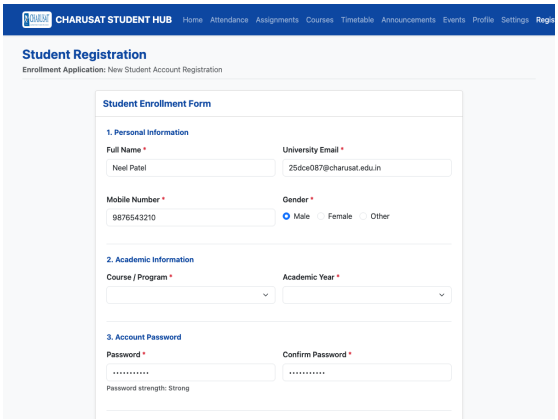


Figure 2: Valid Registration Submission with Real-Time 'Password strength: Strong' Meter

D. Repository Link

GitHub Repository: https://github.com/neel92654/WDF_A_ARP

E. Learning Outcomes

I learned how to construct accessible forms and implement real-time client-side validation using regular expressions, password complexity meters, and user-friendly inline feedback.